CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

Practical Aim:

To implement linear and binary search using both iterative and recursive approaches.

Problem 1 :

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-2

Output:

```
Enter a size:
5
Enter array element:
11
43
56
76
34
Enter a target element:
56
Target is at index: 2
PS C:\Users\HP\OneDrive\Documents\DSA>
```

Problem Faced:

The main challenge in this problem is searching for the target license plate efficiently while implementing both iterative and recursive approaches. Another challenge is handling cases where the target plate may appear multiple times in the list and ensuring that the returned position is consistent.

Outcome/Conclusion:

Two search methods are used: iterative and recursive. The iterative method checks each license plate from the beginning until the target is found. The recursive method checks one plate at a time and calls itself to search the remaining list. Both methods return the position of the target when found. If the target appears multiple times, the first occurrence is returned because the search starts from the beginning and stops at the first match.

Problem 2 :

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly.Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-2

Output:

```
Enter a Size:
5
Enter array Element:
11
23
43
56
45
Enter a target:
56
Target is at index: 3
PS C:\Users\HP\OneDrive\Documents\DSA>
```

Problem Faced:

The main challenge in this problem is efficiently locating the target book code without checking every element in the catalog. Another challenge is ensuring that the search space is reduced correctly after each comparison. If the list is not organized properly, the search may produce incorrect results..

Outcome/Conclusion:

Binary Search is used in both iterative and recursive forms. The search begins with the middle element of the sorted list. If it matches the target, its position is returned. If the target is smaller, the search continues in the left half; otherwise, it continues in the right half. This process repeats until the target is found or the search range becomes empty. Both approaches use the same logic but differ in implementation.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

[Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

Sol: If the list is sorted, some plates can be skipped during the search. This allows the use of Binary Search, which checks only part of the list instead of every plate.

[Problem 2] What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

Sol: If the list is not sorted, Binary Search may search in the wrong half and fail to find the target. Therefore, it may give incorrect results.